1. Springboot RestAPI full project. Learning

First in the series.

==========

Installing Mysql using docker .

Install Docker for MacOS.
Open Terminal
Run command to fetch mysql container-

```
docker pull mysql:latest
```

run mysql docker container -

```
docker run --name mysql-container -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -p 3306:3306 -d mysql:latest
```

Start stop mysql container from Docker desktop app.

you can use Mysql CLI using command :

```
docker exec -it mysql-container mysql -uroot -p
```

check docker logs by :

```
docker logs mysql-container
```

WE CAN USE DOCKER DASHBOARD AS WELL FOR LOGS, TERMINAL ETC..
JUST CLICK ON THE SERVICE... CHECK LOGS TAB, CHECK TERMINAL TAB...
YOU CAN LOG IN TO MYSQL BY mysql -uroot -proot.

==========

Created a spring project with following dependencies :

Web
Validation — for validation I guess. (Bean Validation with hibernate validator)
— @NotNull @Size @Email etc comes under this.

mysql
data jpa
thyme leaf — I dont know about this chat gpt told. I guess its for UI. — Not used in this project.
data jpa
data jdbc.

Calls are stored under MyLocalSpring in postman.

- Updated applicaiton properties to access mysql.

-I created an item table in mysql db.
-Then created an enitity Item with same fields name.
-Created. JPA interface which extends JPARepository <Item, Integer>. —> Item entity and integer is its primary key datatype.

- created a rest controller. findAll , findById etcc.. are inbuilt methods of JPA.

- I can also create a service class where I can write all business logic like saving, updating or anyother operation and then call service class methods from controller.

Other Notes :

Explanation of Key Annotations and Their Usage:

1. **@RestController**: This makes the class a REST controller and ensures that each method's return value is automatically serialized into the HTTP response body.

2. **@RequestMapping("/api/items")**: Sets the base URL for all the handler methods in the controller.

3. **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping**, **@PatchMapping**: These map the HTTP methods to the appropriate CRUD operations. For example, @GetMapping is for handling HTTP GET requests.

4. **@RequestParam**: Used to retrieve query parameters. For example, category can be used to filter items by category (though simplified here).

5. **@PathVariable**: Used to retrieve path variables from the URL, such as id in the endpoint /items/{id}.

6. **@RequestBody**: Binds the HTTP request body to a method parameter. This is used for POST/PUT/PATCH requests when sending data (e.g., item details).

7. **@ResponseStatus**: Allows you to specify the HTTP status code for specific responses, such as HttpStatus.CREATED for successful POST requests or HttpStatus.NO_CONTENT for successful DELETE requests.

8. **@ExceptionHandler**: Handles specific exceptions thrown within the controller and maps them to a HTTP response with an appropriate status.

9. **@Valid**: Ensures that the Item object is validated based on annotations such as @NotNull and @Size.

10. **@CrossOrigin**: Configures CORS support, allowing the API to be called from different origins.

=== ======================

@GeneratedValue — Generates id column automatically.

GeneratedValue automatically generates id based on stretegy we use.

@GeneratedValue(strategy = GenerationType.IDENTITY)

2. Explanation of `@GeneratedValue` Parameters

The `@GeneratedValue` annotation supports different strategies to generate primary key values:

java Copy Edit

```java
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

⚡ Available Generation Strategies

Strategy	Description
`GenerationType.IDENTITY`	Relies on the database to auto-generate the primary key (usually for MySQL, PostgreSQL).
`GenerationType.SEQUENCE`	Uses a database sequence to generate the primary key (common in Oracle, PostgreSQL).
`GenerationType.TABLE`	Uses a separate table to generate primary key values.
`GenerationType.AUTO`	Automatically chooses the appropriate strategy based on the database dialect.

For this project, I have used GenerationType.IDENTITY and made changes in mysql table to make id filed auto_increment. For making field autoincrement we have to make that column primary as well.

For sequence :

java Copy Edit

```java
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "item_seq")
@SequenceGenerator(name = "item_seq", sequenceName = "item_sequence", allocationSize =
private Long id;
```

==========

I have used below annotations for patch request

@Modifying annotation :

Used to indicate that a query modifies the database (e.g., UPDATE, DELETE, INSERT operations).

Works with the @Query annotation in Spring Data JPA.

tells Spring Data JPA that this query should not return a result but modify the database.

The method returns an int representing the number of affected rows.

@Modifying requires @Transactional annotation to handle transaction correrctly.

@Transactional can be used on method or at controller which is calling this method.

@Query annotation :

@query annotation is used to pass our own query.

Defines custom SQL or JPQL (Java Persistence Query Language) queries.

JPQL syntax : @Query("SELECT i FROM Item i WHERE i.name = :name")

SQL way : @Query(value = "SELECT * FROM item WHERE name = :name", nativeQuery = true)

use JPQL to make it database independent.

✅ **JPQL vs. Native SQL:**

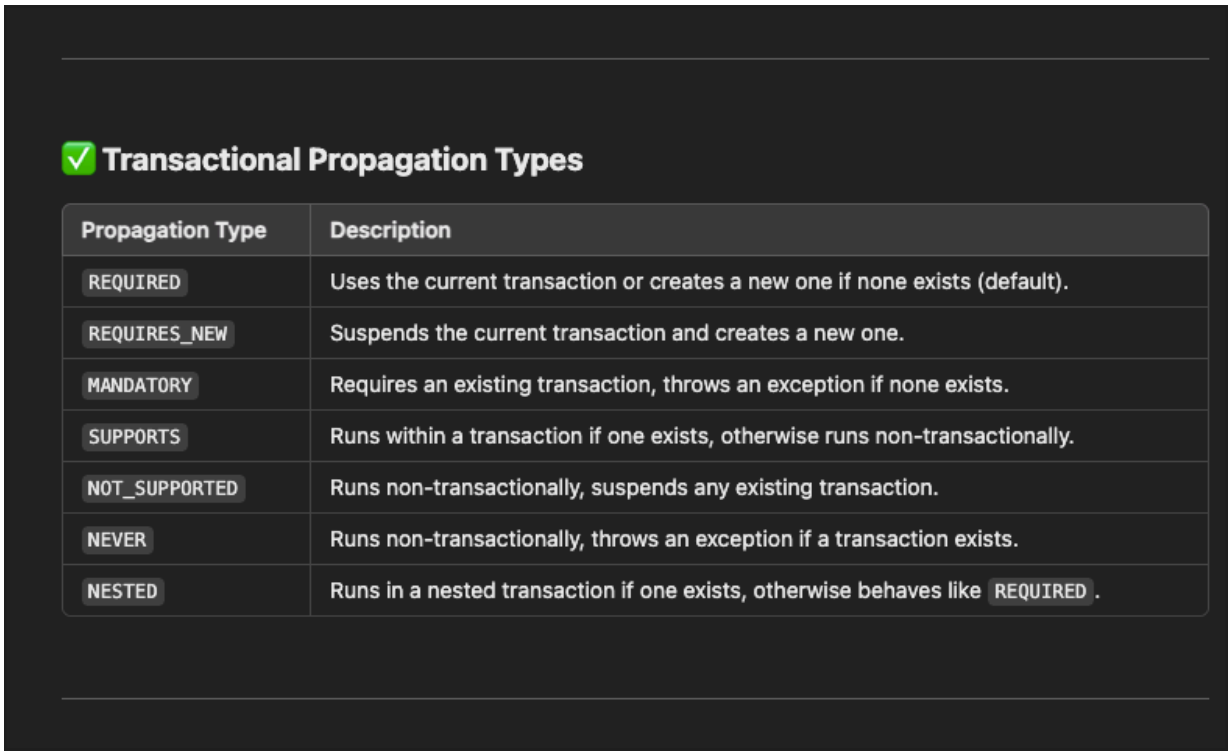
Query Type	Syntax	Usage
JPQL	Java-like entity syntax	Portable, ORM-aware
Native SQL	Database-specific syntax	Direct DB interaction

@Transactional :

By default, it rolls back the transaction **only** for unchecked exceptions (RuntimeException and Error).

To roll back for checked exceptions (Exception), explicitly specify it:

@Transactional(rollbackFor = Exception.class)



✅ Transactional Propagation Types

Propagation Type	Description
`REQUIRED`	Uses the current transaction or creates a new one if none exists (default).
`REQUIRES_NEW`	Suspends the current transaction and creates a new one.
`MANDATORY`	Requires an existing transaction, throws an exception if none exists.
`SUPPORTS`	Runs within a transaction if one exists, otherwise runs non-transactionally.
`NOT_SUPPORTED`	Runs non-transactionally, suspends any existing transaction.
`NEVER`	Runs non-transactionally, throws an exception if a transaction exists.
`NESTED`	Runs in a nested transaction if one exists, otherwise behaves like `REQUIRED`.

==========

@ResponseStatus annotation.

Without @ResponseStatus — all request just retruns 200OK. to return particular response code we should use response status annotation.

@ResponseStatus is used to specify the **HTTP status code** that should be returned by a controller method or exception handler.

✅ Commonly Used Status Codes

HTTP Status	`HttpStatus` Enum	Description
`200 OK`	`HttpStatus.OK`	Successful request
`201 Created`	`HttpStatus.CREATED`	Resource created successfully
`204 No Content`	`HttpStatus.NO_CONTENT`	No content to return
`400 Bad Request`	`HttpStatus.BAD_REQUEST`	Client sent invalid request
`401 Unauthorized`	`HttpStatus.UNAUTHORIZED`	Authentication required
`403 Forbidden`	`HttpStatus.FORBIDDEN`	Access denied
`404 Not Found`	`HttpStatus.NOT_FOUND`	Resource not found
`409 Conflict`	`HttpStatus.CONFLICT`	Conflict with current resource state
`500 Internal Server Error`	`HttpStatus.INTERNAL_SERVER_ERROR`	Server encountered an error

==========

What is @Valid?

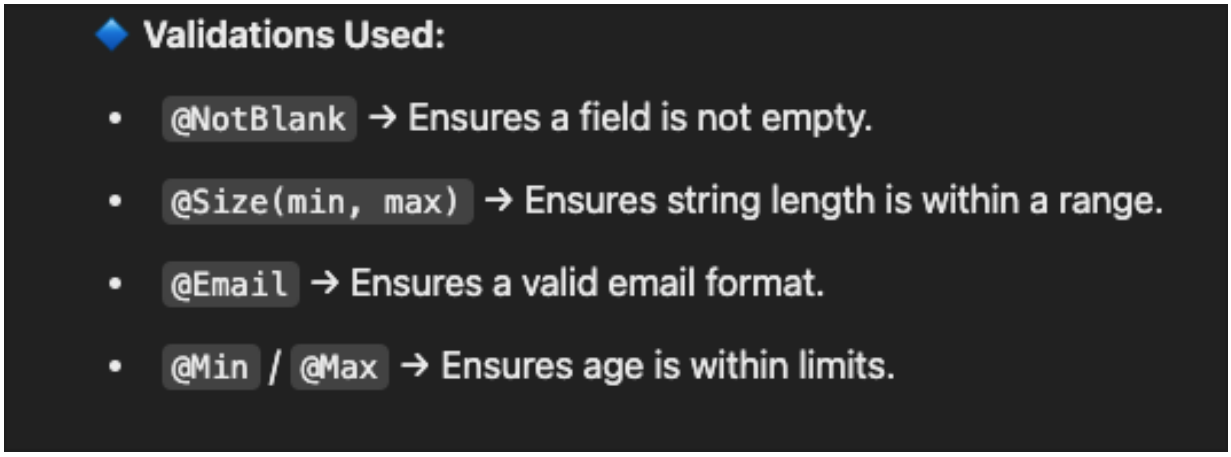
- The @Valid annotation is used in **Spring Boot and JPA** to trigger **bean validation**. It ensures that the incoming request data meets the **validation constraints** defined in the **entity/model class**.
- @Valid ensures validation is applied to the User object inside @RequestBody. **@Valid can only be applied to @RequestBody only. It works with only complex object (java beans) and not primitive type like int**
- If validation **fails**, Spring will return a **400 Bad Request** with validation errors.

I tried by applying @NotBlank(message="name should not be blank") validation over name in entity and also applied @Valid while creating an item in controller. it throws below error if I do not pass name in payload.

2025-04-01T12:06:12.412-04:00 WARN 2907 --- [allannotations] [nio-8080-exec-1] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for argument [0] in public com.restapi.allannotations.bean.Item

com.restapi.allannotations.controller.ItemController.createItems(com.restapi.allannotations.bean.Item): [Field error in object 'item' on field 'name': rejected value [null]; codes [NotBlank.item.name,NotBlank.name,NotBlank.java.lang.String,NotBlank]; arguments [org.springframework.context.support.DefaultMessageSourceResolvable: codes [item.name,name]; arguments []; default message [name]]; default message [Name cannot be blank]]]

Other Validation :



As discussed above @Valid only works with java beans... for primitive type and for @RequestParam and @PathVariable we can annotated class with **@Validated,** this helps in introducing validation at controller level for @RequestParam and @PathVariable.

I have used @Min(1) for deletebyId and also to getbyqueryparam if I use less than 1 then it gives an error in eclipse.

jakarta.validation.ConstraintViolationException: deleteById.id: must be greater than or equal to 1

jakarta.validation.ConstraintViolationException: getItemByIDQueryParam.id: Should be greater than 3

==========

ResponseEnity, annotation.

ResponseEntity<T> is a **wrapper class** in Spring Boot that provides **fine-grained control** over the HTTP response, including:

- **Response Body** (T)
  HTTP Status Code
  Response Headers

Using ResponseEntity, you can **customize API responses** dynamically rather than relying on default Spring responses.

I have created response entity section in eclipse controller.

1. just setting String hello in response Entity.
2. sending Httpstatus along with actual body.
3. sending status, actual body as well as message along with that. Here we are just create a wrapper class which accepts one string and one object..it can be any item, user etc...

Why Place ApiResponse.java in dto/?

- **Separation of Concerns**: Keeps DTOs separate from models and services.

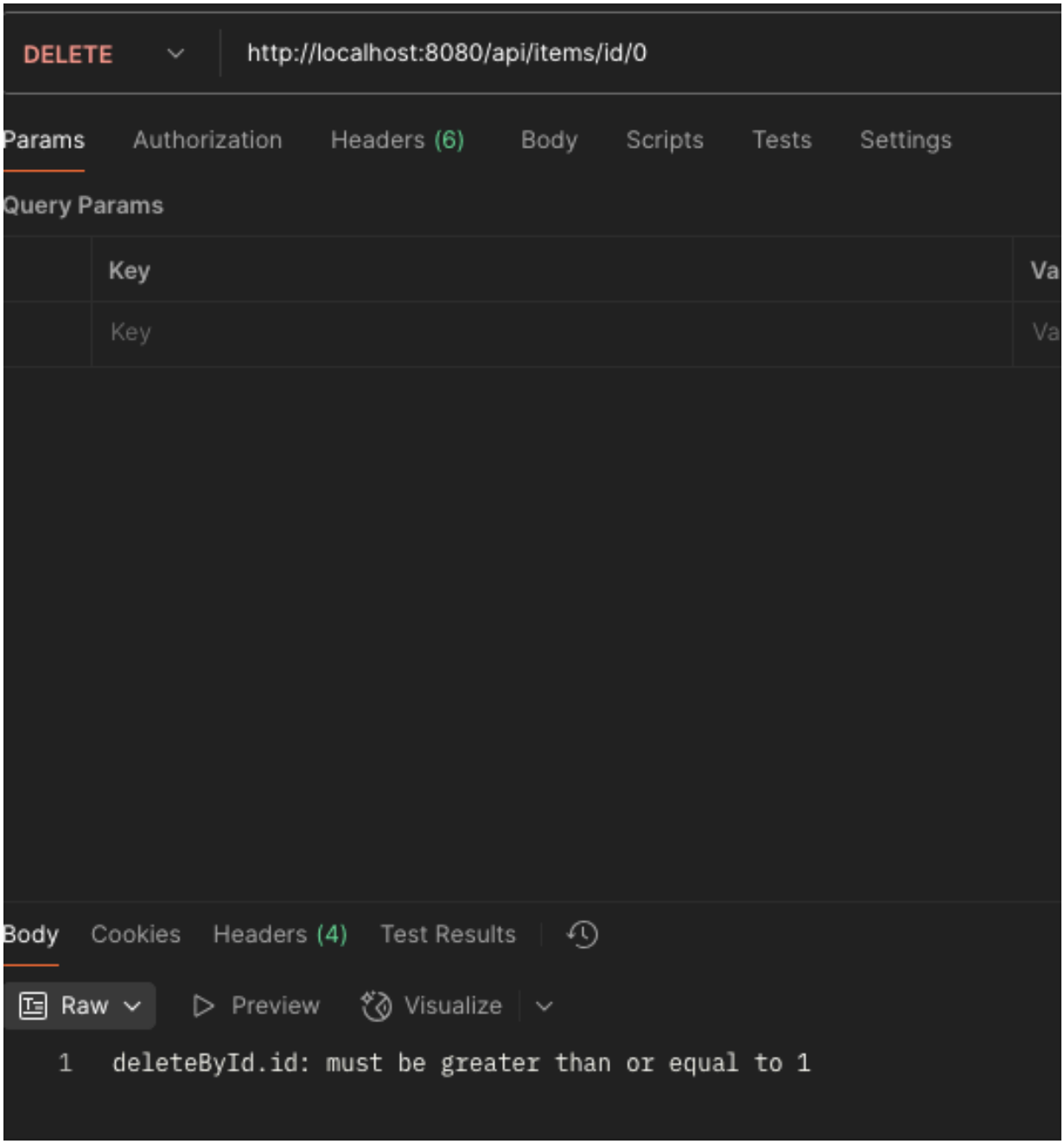
- **Better Code Maintainability**: Centralized place for all API response structures.

- **Standardization**: Helps ensure all responses follow a uniform format.

==========
Global Exception handling in spring - @RestControllerAdvice & @ExceptionHandler annotatins.

Any exception thrown from anywhere in the spring applicaiton can be caught by **@ExceptionHandler**(AnyExceptionName.class)
Where as AnyExceptionName can be Exception, ContraintViolation, IllegalArgumentException, RunTimiException etc...

I first check thed @Min(1) in deleteById method was throwing ConstraintVilationException as mentioned above... So I just created a GlobalExceptionHandler class and annotated it with @RestControllerAdvice and annotated particular method with **@ExceptionHandler(ConstraintViolationException.class)** and boom, all set... I was able to see my message in a response without doing anything.

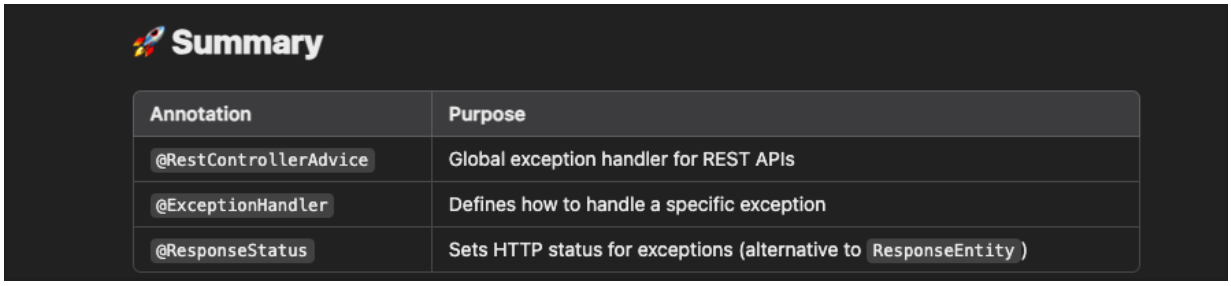


I can also throw exception manually from anywhere and can configure global

exception class to handle that exception.

@RestControllerAdvice in Spring Boot

@RestControllerAdvice is a **global exception handling mechanism** in Spring Boot for REST APIs. It allows you to **handle exceptions across the entire application** in one centralized place instead of writing try-catch blocks in every controller.
It is a specialization of **@ControllerAdvice** but specifically designed for REST controllers by combining it with @ResponseBody.



🚀 Summary

Annotation	Purpose
`@RestControllerAdvice`	Global exception handler for REST APIs
`@ExceptionHandler`	Defines how to handle a specific exception
`@ResponseStatus`	Sets HTTP status for exceptions (alternative to `ResponseEntity`)

Basically there are three ways to handle exception in spring application .

1. Try Catch .. Where you catch exception and set ResponseEnitity to send response accordingly

```
    @GetMapping("/{id}")
    public ResponseEntity<?> getItem(@PathVariable Long id) {
        try {
            Item item = itemService.getItemById(id)
                    .orElseThrow(() -> new RuntimeException("Item not found"));
            return ResponseEntity.ok(item);
        } catch (Exception ex) {
            return ResponseEntity.status(404).body("Error: " + ex.getMessage());
        }
    }
}
```

2. Global exception handling as discussed above.
3. Exception Handling for Validation Errors (@Valid) — as discussed above.

==========

Handling of large data via Spring Rest API

Strategies to Handle Large Data Transfer in Spring REST API — ...

RESPONSE STREATEGIES. ....

1️⃣ **Use Pagination (Best Practice). ——> I have implemented finaAll method in Repository with outputs page... call in controller is / paginated.**
Instead of fetching **millions of records at once**, fetch data **page by page**.

2️⃣ **Use Streaming for Large Data Output**
For large JSON responses, **stream data** instead of loading everything into memory.

Streaming is useful when returning **large datasets** without loading them all into memory at once. Instead of sending the **entire response** at once, data is sent **in chunks** as it's processed.

I have implemented new api /stream. I have used StreamingResponseBody interface and override writeTo method.

3️⃣ **Use Compression (GZIP)**
Reduce **data transfer size** using **GZIP compression**.

Spring Boot can automatically compress large responses using **GZIP**.

✅ Enable GZIP Compression in application.properties

```
server.compression.enabled=true
server.compression.mime-types=application/json,application/xml,text/plain
server.compression.min-response-size=1024
```

REQUEST STREATEGIES. ....

4️⃣ **Use Asynchronous Processing**

For large uploads, **use async processing** and return a task ID instead of blocking.

@Async Example: Asynchronous Processing in Spring Boot
This **saves an item in the database** asynchronously. It immediately gives response to the user and does task asynchronously using separate thread.

We can tag a method which we accepts request for processign with **@Aync annotation.**
Without **@EnableAsync,** Spring **won't recognize** @Async methods, and they will execute **synchronously** instead of asynchronously.

We can either declare @EnableAysnc in configuration class or in main springboot class below @SpringBootApplication.

I have created an Aync post method. also created a service class and made service class method async execution.

here we are using thenAccept method. which will not wait for the result but whenever result is available it will just print
that result. below sysout "Async task completed with result: Items saved successfully!" gets logged in eclipse once thread
sleep is done. But actual response that is "Item creation started asynchronously!" printed in postman immediately.
 for user to know the status of request, we can implement polling mechanism for them.. we can pass taskid to the user
using which it can poll the result. Not implemented here. Its easy but. can as chat gpt.
Other method instead of polling which we can use for client to know about status of request are : Websocket and callbackurl

polling can be as simple as creating two methods to pass task id : and then creating an api using which client can poll the result of request.

```
    public Long startTask(List<Item> items) {
        long taskId = taskCounter.incrementAndGet();
        // Start async task and return taskId
        processItemsAsync(items, taskId);
        return taskId;
    }

    public String getTaskStatus(Long taskId) {
        // Here you can add logic to check whether the task has completed
        // For this example, we're simulating that all tasks complete after a delay
        return "Task " + taskId + " is in progress..."; // Or return "completed" once
    }
}
```

During response we are passing taskid as well.. its not visible here..it got cut.

```
    @PostMapping("/async")
    public ResponseEntity<String> createItems(@RequestBody List<Item> items) {
        Long taskId = itemService.startTask(items);  // Start the task and get taskId
        return ResponseEntity.accepted().body("Item creation started asynchronously!
    }

    @GetMapping("/status/{taskId}")
    public ResponseEntity<String> getStatus(@PathVariable Long taskId) {
        String status = itemService.getTaskStatus(taskId); // Check the task status
        return ResponseEntity.ok(status);
    }
}
```

Polling: The client can periodically check the status of the task.
WebSockets: The server can push updates to the client in real-time.
Callback URL (Webhooks): The server notifies the client when the task is complete.

5️⃣ Use Chunked Uploads for Large Files

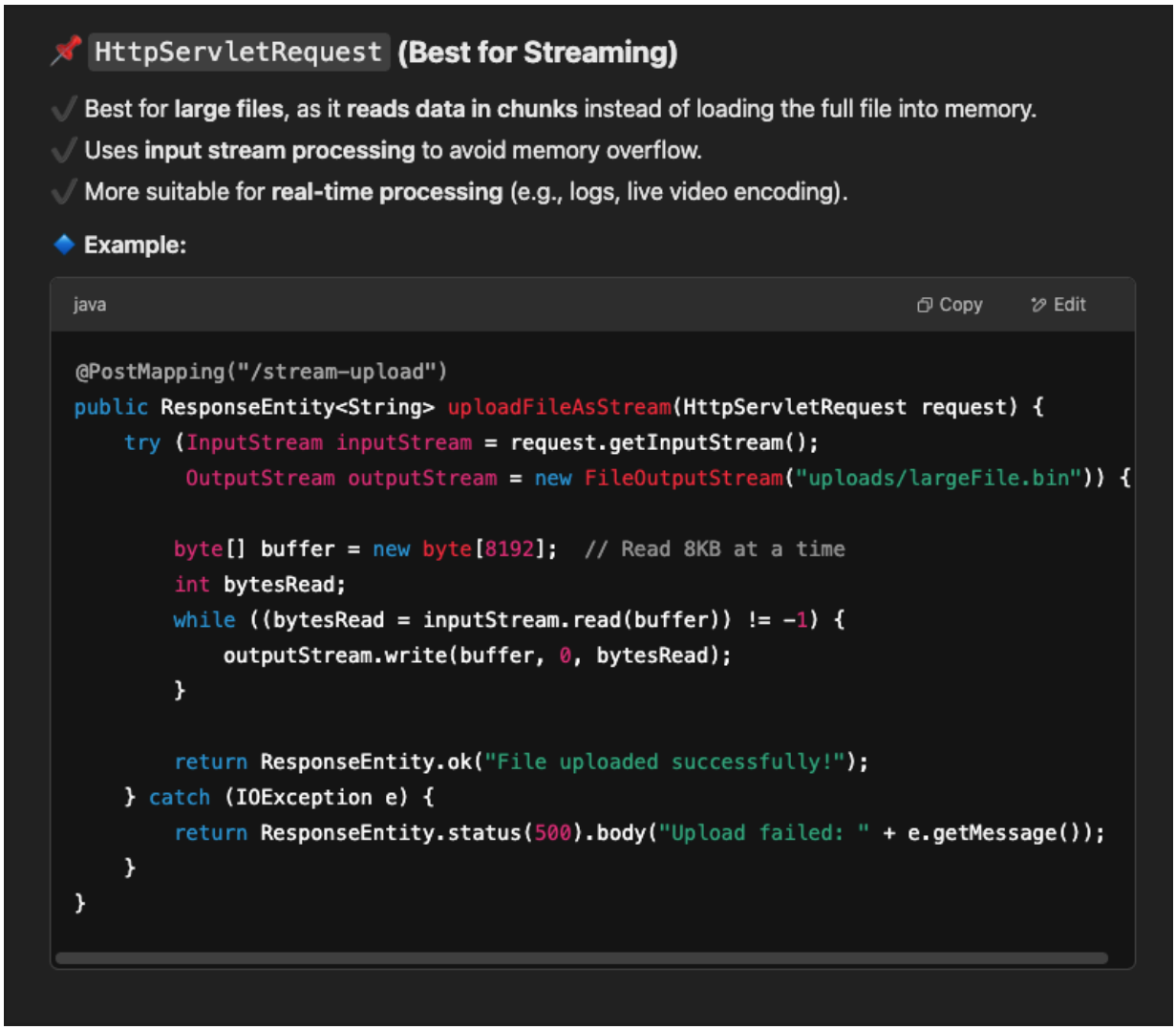
Split large files into chunks and **upload in parts**.

I have not done practical for this but got information from chatgpt as follow.

For largefile upload, either we can divide that file into **chunk or use streaming to upload file.** or for small file we can directly **accept full file**

using @RequestParam(“file”) Multiplart file.

Streaming Example api endpoint :



📌 `cURL` Command to Upload a File as a Stream

sh Copy Edit

```sh
curl -X POST "http://localhost:8080/stream-upload" \
     -H "Content-Type: application/octet-stream" \
     --data-binary "@largefile.bin"
```

📌 Explanation of cURL Command

Flag	Description
`-X POST`	Sends a **POST** request.
`-H "Content-Type: application/octet-stream"`	Tells the server that **raw binary data** is being sent.
`--data-binary "@largefile.bin"`	Reads the file `largefile.bin` and **sends it as a continuous stream.**

For chunking logic :

We create an endpoing like below :

```java
@PostMapping("/chunk")
public ResponseEntity<String> uploadChunk(@RequestParam("file") MultipartFile chunk,
                                          @RequestParam("fileName") String fileName,
                                          @RequestParam("chunkIndex") int chunkIndex,
                                          @RequestParam("totalChunks") int totalChunks
```

User devides file and upload in a chunk...

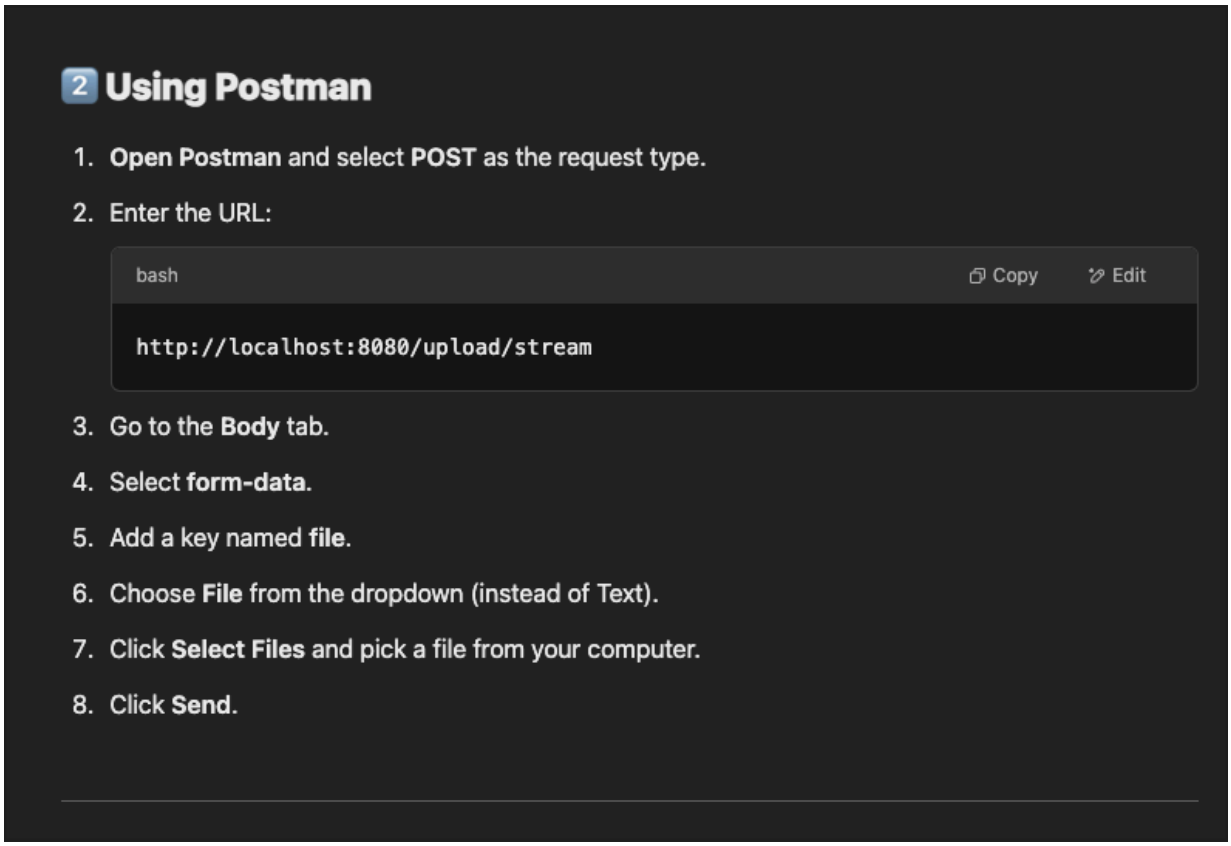
Uploading File in Chunks

bash Copy Edit

```bash
# Upload chunk 1 (Assume file is split into 3 chunks)
curl -X POST "http://localhost:8080/upload/chunk" \
     -F "file=@chunk1.bin" \
     -F "fileName=largeFile.bin" \
     -F "chunkIndex=0" \
     -F "totalChunks=3"

# Upload chunk 2
curl -X POST "http://localhost:8080/upload/chunk" \
     -F "file=@chunk2.bin" \
     -F "fileName=largeFile.bin" \
     -F "chunkIndex=1" \
     -F "totalChunks=3"

# Upload chunk 3 (Final chunk)
curl -X POST "http://localhost:8080/upload/chunk" \
     -F "file=@chunk3.bin" \
     -F "fileName=largeFile.bin" \
     -F "chunkIndex=2" \
     -F "totalChunks=3"
```



In business logic, we use RandomAccessFile and we write all the chunks directly into final file on server side... so merging is automatically.
Alternatively,
We can also accept each chunks and later merge it.

For file upload, we can use @RequestParam("file") MultipartFile file.

Normal File upload without streaming and chunking:

📌 `@RequestParam("file") MultipartFile` (Best for Regular Uploads)

✔️ Best when **handling smaller files (e.g., <100MB)**.

✔️ Loads the **entire file into memory** before processing, so **not ideal for very large files.**

✔️ Supports **file metadata** (original filename, size, etc.).

🔷 **Example:**

java Copy Edit

```java
@PostMapping("/multipart-upload")
public ResponseEntity<String> uploadFile(@RequestParam("file") MultipartFile file) {
    try {
        file.transferTo(new File("uploads/" + file.getOriginalFilename()));
        return ResponseEntity.ok("File uploaded successfully!");
    } catch (IOException e) {
        return ResponseEntity.status(500).body("Upload failed: " + e.getMessage());
    }
}
```

Feature	Streaming Upload (Direct Stream Processing)	Chunked Upload (Client-Side Chunking)
How It Works	The file is sent as a **continuous stream** without loading it fully in memory.	The file is **split into smaller chunks** by the client and sent in multiple requests.
Server-Side Handling	The server processes the file **as it receives data** (e.g., writing to disk, processing).	The server **stores chunks first**, then merges them into a full file after the last chunk arrives.
Resumability	❌ **No** – If upload fails, the entire file must be reuploaded.	✅ **Yes** – Only failed chunks need to be reuploaded.
Memory Usage	✅ **Low** – Since data is processed in real-time, **no need to store the full file in RAM**.	✅ **Low** – Since chunks are stored separately, the entire file doesn't load into RAM.
Timeout Issues	✅ **Lower risk** – As data is streamed, server doesn't wait for full file before processing.	✅ **Lower risk** – Since chunks are uploaded separately, **long files don't cause timeout issues**.
Client Complexity	✅ **Simple** – Client sends file **as a single stream**, no chunking logic needed.	❌ **More complex** – Client must **split the file** into chunks, track progress, and retry failures.
Server Complexity	✅ **Simple** – Just **read the stream** and save/process the data.	❌ **More complex** – Requires **chunk tracking, merging, and cleanup**.
Best For	**Streaming large files for direct processing** (e.g., video live processing, logs, real-time data)	**Large file uploads with resumability** (e.g., cloud storage, resumable uploads)

🚀 For Efficient Large File Streaming:

If you're dealing with **very large files** and want to avoid request timeouts:

- Use `chunked transfer encoding` (`Transfer-Encoding: chunked`) if your server supports it.
- Use `WebSockets` or `gRPC` for real-time streaming instead of raw HTTP.

Would this approach work for your use case?

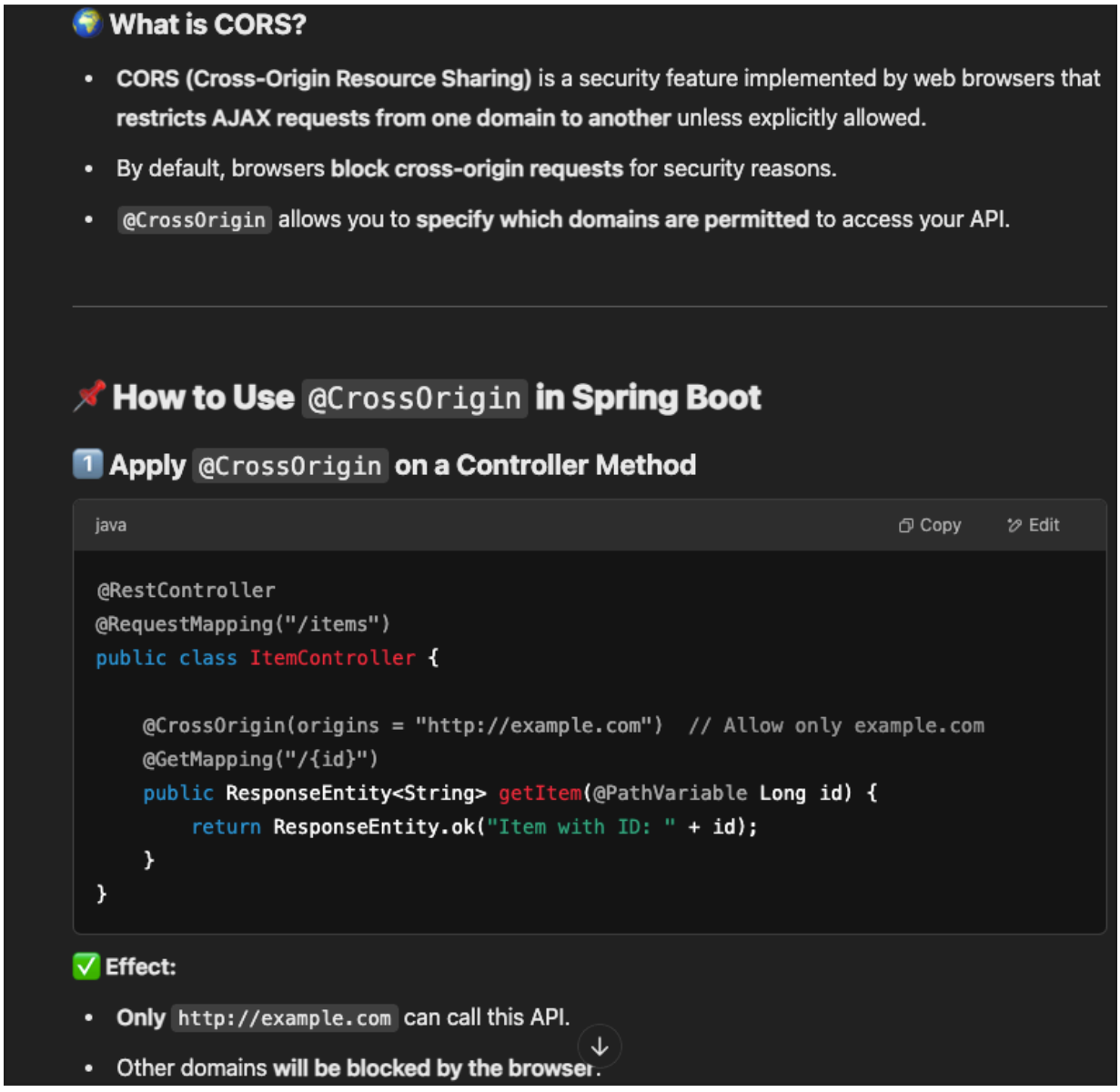
==========

@CrossOrigin annotation — CORS

@CrossOrigin in Spring Boot

The @CrossOrigin annotation in Spring Boot is used to **enable Cross-Origin Resource Sharing (CORS)** for RESTful APIs.

@CrossOrigin(origins = "http://example.com"). This annotation can be placed over the controller class or over the method of controller class.
Only request from this domain will be served. If its placed over the controller class then all request inside that conroller will be only served to the request if its originated from this URL.



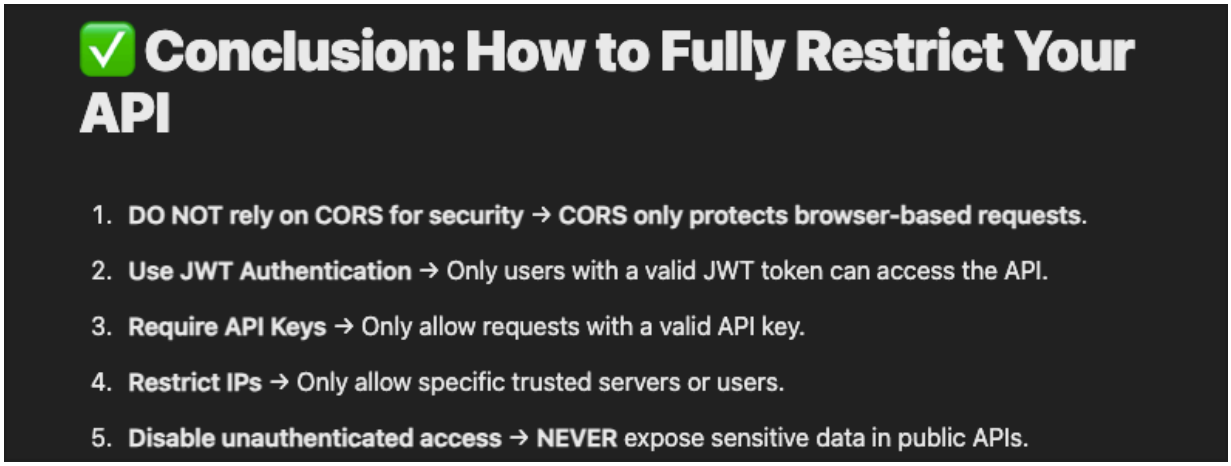
Syntax :

@CrossOrigin(origins = “*”). — It allows all domains
@CrossOrigin(origins = {"http://example.com", "http://anotherdomain.com”}).
— Multiple domains.

Instead of controller level or method level, we can also configure CORS at global level using **WebMvcConfigurer class. Did not go in deep for this.**

CORS is only enforced by browser. Its not enforced my cURL or Postman. I tried feature of @Crossorigin by following.

**I have created a new api end point /corstry. Allowed only example.com..
From postman while calling this I modified header and introduced Origin with value as http://localhost.
Its showing not allowed. If I change origin to example.com then it allows request.
Its not 100% secure as anybody can still call my endpoing using curl or postman. Only browser enforces it. So other security measures needs to be at its place like.**



==========

I have created a separate file for Spring AOP. But kept practicle in the same project com.restapi.allannotations….

==========

Next topic :

Next topic : start with other annotations in above main list.
@AuditableMethod
Using @ControllerAdvice for Global Exception Handling